

ELEE 4230/6230: Sensors and Transducers

Course Project – Summer 2026

‘Sourdough Tracker’

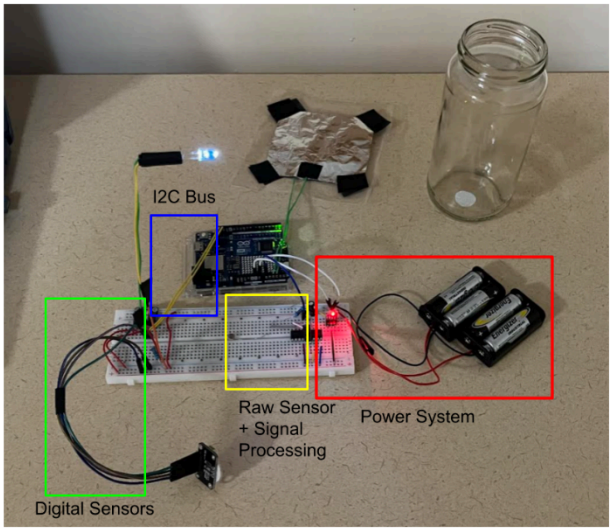
Shruthi Rajamanikandan

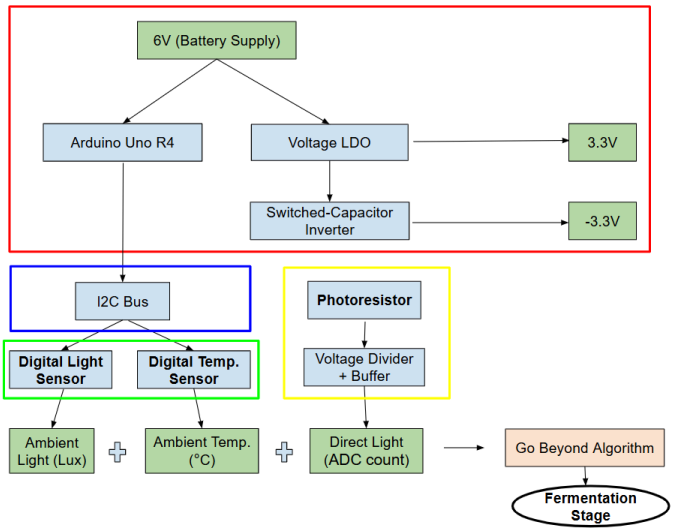
Table of Contents

ELEE 4230/6230E: Course Project 1

0. A Tour of ‘Sourdough Tracker’ 2
1. Problem Definition and Environment and Project Constraints 3
2. Sensor Selection and Design Decisions 5
3. RAW Sensor Dynamics and Signal Processing 7
4. Capacitive Sensor (User Input System) 11
5. Other Sensors Used 14
6. Digital Logic + Algorithms 16
7. Experimental Results 18
8. Failure Analysis 20
9. PCB Design 21
10. Course Reflection 22
11. References 23
12. Appendix 24

0. A Tour of ‘Sourdough Tracker’





The system is powered by a 6V battery supply, which splits into two paths. One path feeds the Arduino Uno R4 directly, while the other passes through a voltage LDO regulator that produces a clean 3.3V rail; this 3.3V is routed through a switched-capacitor inverter to generate a complementary -3.3V rail (needed for the LM358 op-amp). The Arduino Uno R4 communicates over an I2C bus with two digital sensors — a digital light sensor and a digital temperature sensor — which report ambient light (in lux) and ambient temperature (in °C) respectively. The touch sensor allows the user to reset data collection. In parallel, a photoresistor is powered by the ±3.3V rails through a voltage divider and buffer circuit, converting changes in direct light intensity into an analog voltage that the Arduino reads as an ADC count. These three measurements: ambient light, ambient temperature, and direct light are analyzed through the "Go Beyond" algorithm.

Algorithm Overview: The raw photoresistor ADC count is first normalized against the ambient lux reading to cancel out drift from changing room light conditions, isolating the portion of the signal caused only by the dough rising and blocking the light; this normalized value is then converted into a rise percentage using calibrated baseline (initial flat dough) and fully-risen reference points, and the rate of change of this rise percentage over time is calculated to capture how quickly fermentation is progressing; this rate is further corrected using the ambient temperature reading, since fermentation speeds up in warmer conditions and slows in cooler ones. Finally, the algorithm classifies the current state into one of 3 fermentation stages: **Rising, Peak, or Over-proofed** based on the combination of rise percentage and temperature-corrected slope.

1. Problem Definition and Environment and Project Constraints

Problem: Home bakers proofing sourdough currently rely on subjective visual and tactile cues: eyeballing volume, poking the surface, and guessing from experience to judge when dough has finished rising. This is unreliable, since the rise rate depends on flour quality, ambient temperature, and sourdough ratio, all of which vary batch to batch and kitchen to kitchen. Over- or under-proofed dough produces poor bread structure, and the only way to check with certainty today is to disturb the dough (lifting a cloth, opening a proofing box), which itself disrupts the rise and risks contaminating or deflating the dough.

The problem stated objectively is: **How can rise progress be determined non-invasively, continuously, and accurately, in an uncontrolled home kitchen environment?**

Environment: The device operates on an open kitchen counter or inside a proofing box, not a controlled lab. Ambient light varies with time of day, kitchen lighting, and momentary occlusion (someone walking past, opening a nearby cabinet). This directly interferes with the primary optical growth signal. Temperature: kitchens experience drafts, oven/appliance heat, and seasonal swings; the device's own electronics generate localized heat that can bias a sensor. Need to consider no reliable outlet near many counter locations, so the device must run on battery for the multi-hour duration of a proof. The sensing hardware sits outside the jar and must not contact the dough or be positioned where condensation/flour dust could compromise it.

The safety checklist confirms rail voltages stay within safe voltage limits, the LDO keeps components within safe thermal bounds, the enclosure has no exposed hazards, and no sensing hardware directly contacts food. The main flagged item was securing battery orientation in the AA holder with an enclosed compartment.

Project-specific constraint	Why it exists	Design decision it drove
Kitchen ambient light varies constantly (day/night, foot traffic, light switches)	Uncontrolled home environment	Added ambient light sensor as a Kalman filter reference to separate lighting noise from real growth
MCU/regulator generate localized heat	Onboard component power dissipation	Temperature sensor physically separated from rest of electronics on the layout
Device sits directly next to food	Food-contact safety	No sensor placement inside the jar. I had to place components to make space for the jar.

Jar material/clarity isn't standardized	User may substitute containers	Algorithm will read the initial opacity of the flat dough to calibrate for different jar shapes
Materials had to be ordered based on price and arrival times	Time and Money and Deadlines	Sensors are cheap and not element-proof. I had to sacrifice more accurate readings especially for temperature.



(AI Generated; Prompt: Final Revision Idea)

Use case: The user can see how far along in the fermentation process their starter is since they fed it. When the dough hits “Peak” it is signaled as ready to the user. The user can immediately bake with the dough to achieve the perfect baked good (no under or over proofing).

Limitations:
As sourdough ferments, it releases moisture and heat, creating condensation on the inside of the lid and upper jar walls. Droplets forming directly over the light source or sensor can scatter the light beam, causing inaccurate lux readings even if the dough hasn't moved.

Top-down light transmission measures overall path density rather than exact 3D physical volume. A starter that domes significantly in the center vs. one that rises flat might read the same optically, making it harder to pinpoint the exact peak volume.

2. Sensor Selection and Design Decisions

Category	Sensor Option	Pros	Cons
Opacity (Raw Light)	Photoresistor [Chosen]	Cheap & simple; Highly sensitive to light changes; Fast analog response	Highly sensitive also means susceptible to ambient room light
Opacity (Raw Light)	Photodiode	Extremely linear response; Fast response time	Requires active op-amp current-to-voltage conversion; Tiny output signal
Temperature (Digital)	PCT2075 [Chosen]	Relatively small for a digital sensor; Built-in over-temp alert pin	Not resistant to elements (water + dust from the kitchen)
Temperature (Digital)	DS18B20	Waterproof; Prewired	Slower 1-Wire protocol; More coding work required
Ambient Light	TSL2591 (Digital)	Measures exact Lux values; High dynamic range	Overkill for relative opacity; Takes up an additional I2C address
Ambient Light	VEML7700 [Chosen]	Similar to light readings of the human eye; Cheap	Lower max lux range than TSL2591; Requires gain/integration time tuning for accurate readings

Photoresistor: chosen as the raw opacity sensor. It's cheap, fast, and highly sensitive to the small changes in light transmission through the jar as dough rises and the mixture becomes less/more opaque. This directly satisfies the system's main requirement: a continuous, non-invasive measurement of dough growth.

PCT2075: It's compact enough to fit on the jar and it communicates over I2C, satisfying the requirement for a standard short-distance protocol, and its size made it easier to physically separate from the MCU/regulator on the board layout to avoid self-heating bias.

VEML7700: Its spectral response approximates human eye sensitivity, which matters here because the goal isn't absolute lux measurement, it's determining whether a perceptible lighting change is happening at the same time as a change in the LDR reading. That's exactly the comparison the Kalman filter needs, and the VEML7700's low cost and I2C interface meet the budget and protocol constraints without adding capability the system doesn't use.

Decision 1: Photoresistor for raw opacity sensing

- *Chosen:* Photoresistor (LDR) as the raw, nonlinear light sensor measuring opacity through the jar.
- *Alternative considered:* Photodiode.
- *Why it wouldn't work as well:* The photodiode's output is a very small current that requires an active op-amp transimpedance stage just to produce a usable voltage, adding circuit complexity before any of the project's required signal conditioning even begins. Its highly linear response also removes the nonlinear-transfer-function characteristic the raw sensor needs to have, so it would have pushed the project toward an artificially non-linear stage elsewhere just to meet that requirement, rather than getting it naturally from the sensor itself.

Decision 2: PCT2075 for temperature

- *Chosen:* PCT2075 digital temperature sensor.
- *Alternative considered:* DS18B20.
- *Why it wouldn't work as well:* The DS18B20 is waterproof and prewired, which looks appealing for a kitchen setting, but it communicates over the 1-Wire protocol rather than I2C. Since the other sensor already shares an I2C bus, adding a second protocol would mean more code, and more wires. It was also expensive: $10 dollars compared to around $3 for the PCT2075.

Decision 3: VEML7700 for ambient light reference

- *Chosen:* VEML7700 ambient light sensor.
- *Alternative considered:* TSL2591.
- *Why it wouldn't work as well:* The TSL2591 offers a wider dynamic range and precise lux output, but that precision is unnecessary here since the system only needs a relative signal to tell the Kalman filter "ambient light changed" versus "ambient light is stable," not an accurate absolute brightness reading. The TSL2591 also costs more and, like the VEML7700, occupies its own I2C address, so there's no advantage to offset its higher precision.

3. RAW Sensor Dynamics and Signal Processing

Raw Sensor Physics and Modeling

My project utilizes a CdS photoresistor. In darkness, valence electrons are bound to the lattice, yielding a high baseline resistance $R_{LDR} > 100k\Omega$. Incident photons exceeding the CdS bandgap excite electrons into the conduction band, generating electron-hole pairs that drop resistance via an inverse power law: $R_{LDR} = A*Lux^{B}$. Wired into a voltage divider with a fixed 10kΩ resistor pulled up to the 3.3V LDO rail, the output voltage maps as a non-linear rational function: $Vout = 3.3V * \frac{10k}{R_{LDR} + 10k}$. To optimize the 10-bit ADC resolution without saturation, the region of interest is restricted to an indoor range of 243 Lux to 7,178 Lux.

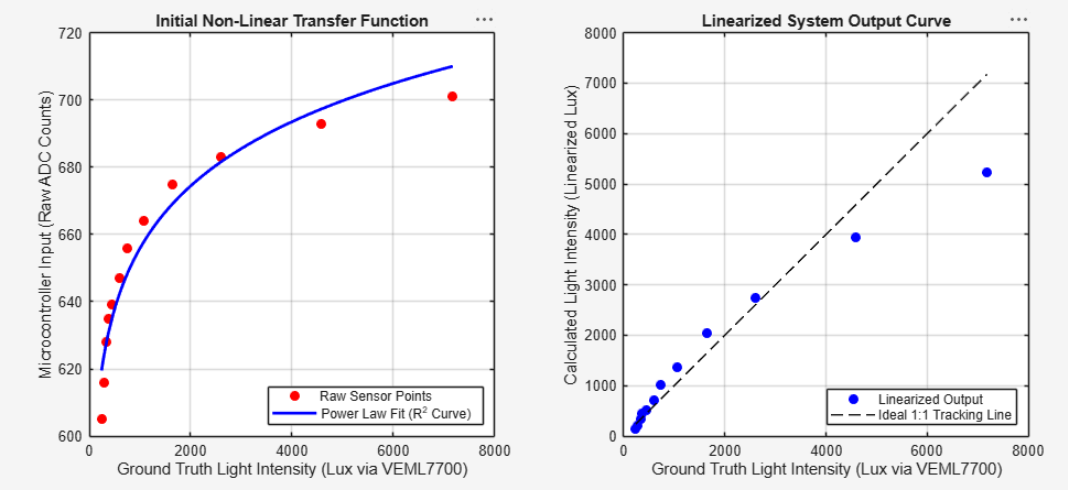
This table includes the raw LDR sensor data compared with the packaged ambient light sensor data as a true value for lux.

Distance (in.)	12	11	10	9	8	7	6	5	4	3	2	1
True Lux	243	287	332	373	443	598	740	1070	1651	2610	4578	7178
Raw ADC step	605	616	628	635	639	647	656	664	675	683	693	701

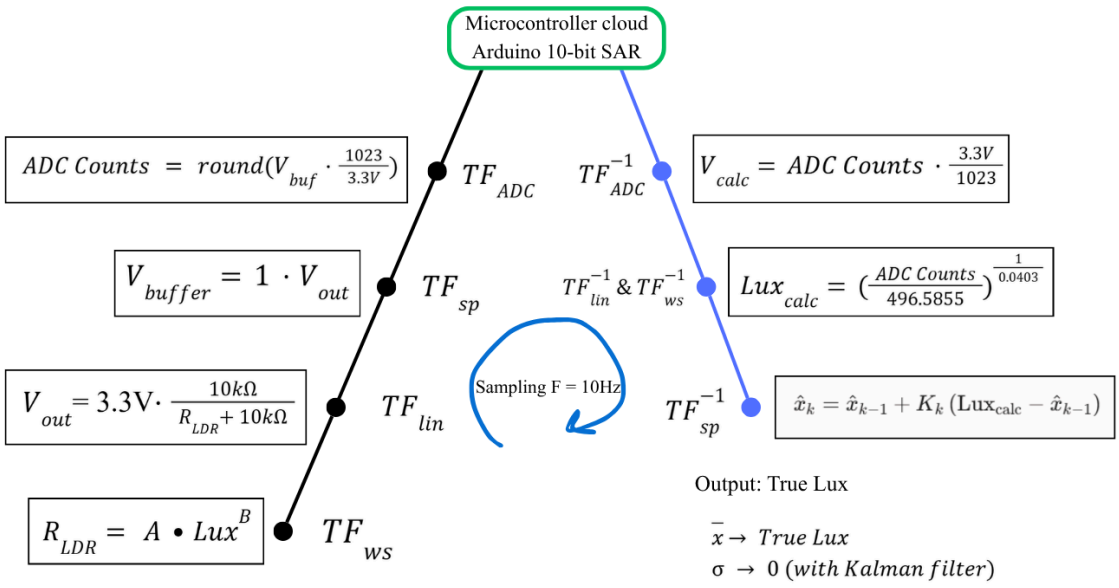
A logarithmic transformation $ln(ADC) = ln(A) + Bln(Lux)$ and linear least-squares regression in MATLAB solved the empirical constants as A = 496.5855 and B = 0.0403, yielding the baseline model $ADC = 456.5855 * Lux^{0.0403}$. To linearize this in software, this model was inverted algebraically to compute real-time lux from the LDR ADC counts:

$$Lux_{calc} = \left(\frac{ADC}{496.5855}\right)^{\frac{1}{0.0403}}$$

The regression model achieved a high coefficient of determination ($R^2 = 0.9100$), proving that the power-law equation accounts for 91% of the circuit variance. The system's RMSE is 618.51 Lux. This error is localized at the upper boundary (>4,000 Lux) where the extreme voltage compression makes the inverse equation highly sensitive to single digital quantization steps. However, within the primary indoor tracking range (10 Lux to 1,000 Lux), the model operates in its high-sensitivity zone with minimal residual error, fully satisfying design accuracy constraints.



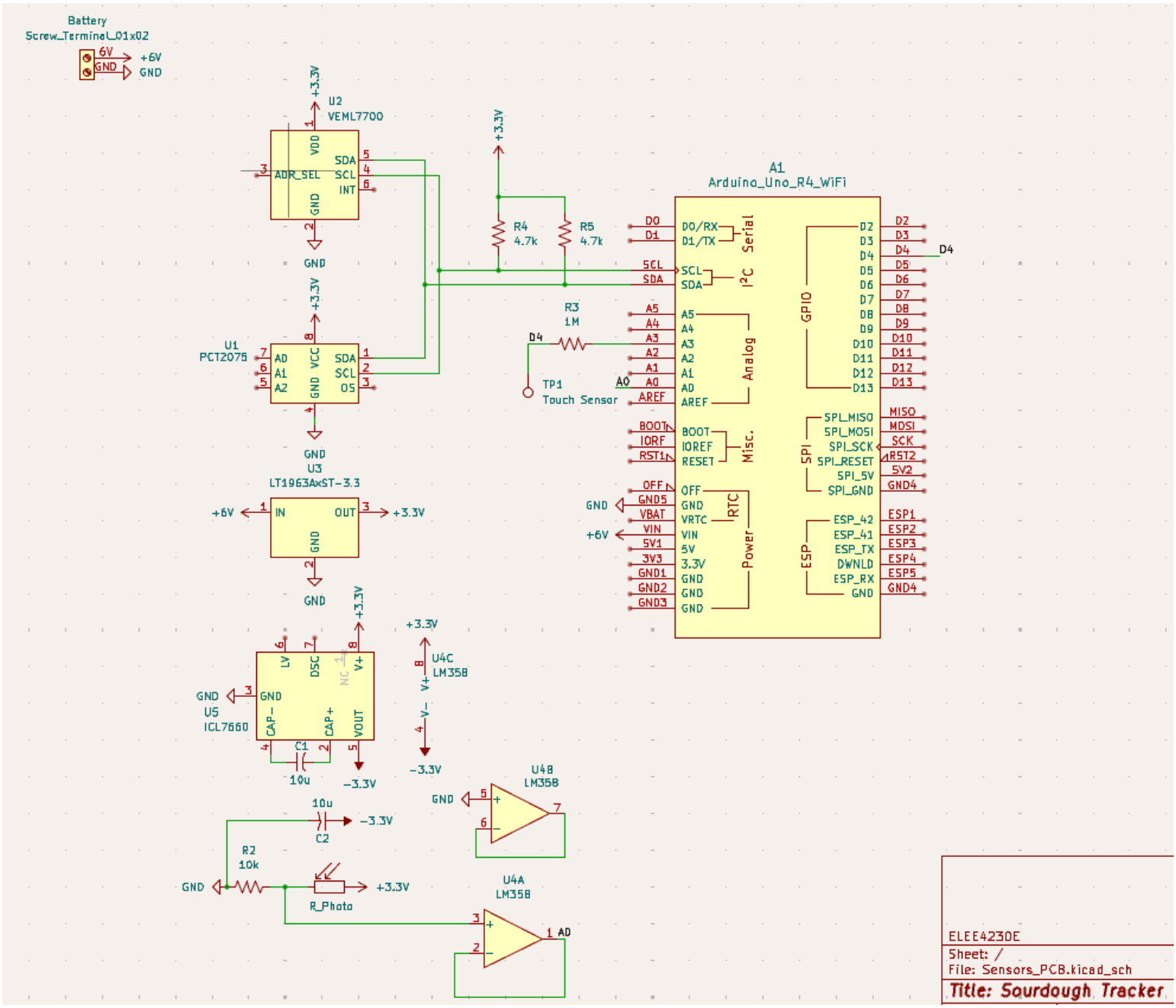
Signal Processing Path ("Signal Processing Mountain")



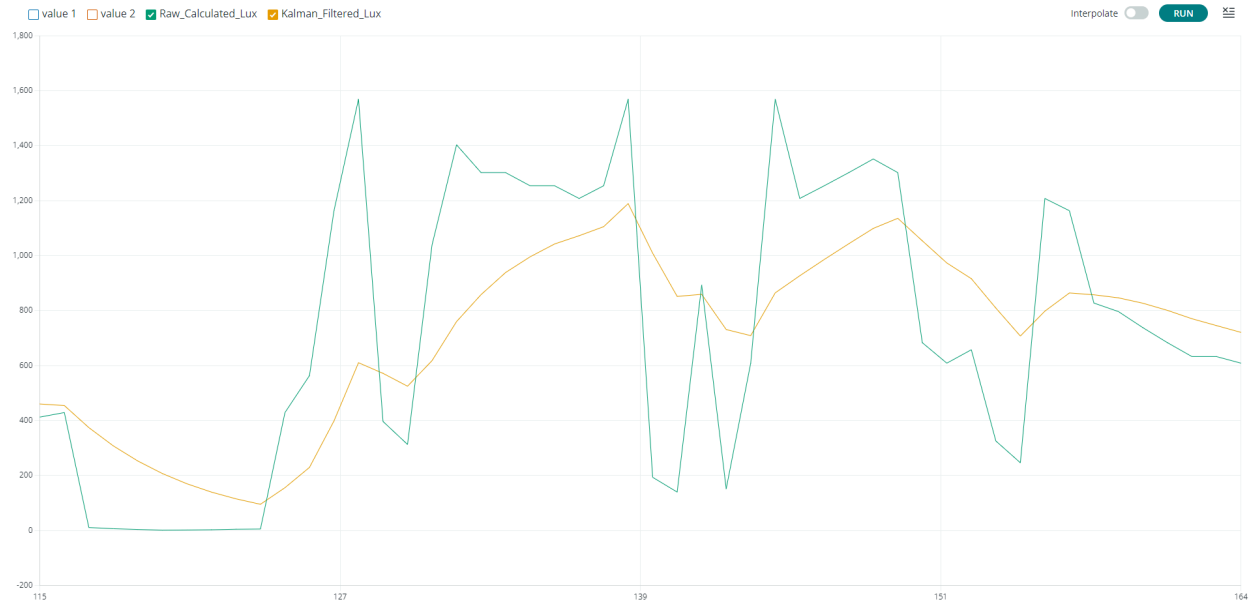
The raw analog signal comes from the center node of the photoresistor voltage divider. To isolate this voltage from loading effects, it is routed into the non-inverting input of an LM358 op-amp that acts as a unity-gain buffer. This active stage presents a near-infinite input impedance (10^{12} Ω) to the divider network, preventing the microcontroller from drawing current. The op-amp also provides an ultra-low output impedance (<1Ω) to drive analog pin A0. System calibration was achieved by coupling this LDR circuit alongside a packaged VEML7700 I2C light sensor on a metric track to sweep light source distances across the 243 to 7178 Lux region of interest to generate the empirical dataset used for baseline linear regression.

Final Implementation (Circuit and Power)

The entire system runs off an isolated 6.4V serial AA battery pack. The raw 6.4V line feeds the Arduino's Vin pin, where an onboard buck regulator steps it down to a 5.0V digital system rail. To safely power the sensors and op-amp, an external 3.3V LDO linear regulator branches off the 5.0V line to supply exactly +3.3V. To get the -3.3V, an ICL7660 charge-pump inverter taps this 3.3V LDO line. This IC mirrors the voltage below zero to power the negative rail pin of the op-amp. The entire system maintains a single, unified common ground of 0V.


Battery
Screw_Terminal_01x02
6V
GND
+6V
GND
U2
VEML7700
VDD
SDA
ADR_SEL
SCL
INT
GND
+3.3V
R4
4.7k
R5
4.7k
A1
Arduino_Uno_R4_WiFi
D0
D1
D0/RX
D1/TX
Serial
SCL
SDA
I2C
GPIO
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
R3
1M
D4
TP1
Touch Sensor
A0
A5
A4
A3
A2
A1
A0
AD
AREF
Analog
U1
PCT2075
AD
A1
A2
VCC
GND
OS
U3
LT1963AxST-3.3
IN
OUT
BOOT
IOREF
RST1
RESET
Misc.
SPI
SPI_MISO
SPI_MOSI
SPI_SCK
SPI_RESET
SPI_5V
SPI_GND
MISO
MDSI
SCK
RST2
5V2
GND4
OFF
GND5
VBAT
VRTC
VIN
5V1
3V3
GND1
GND2
GND3
RTC
Power
ESP
ESP_42
ESP_41
ESP_TX
DWNLD
ESP_RX
ESP1
ESP2
ESP3
ESP4
ESP5
U5
ICL7660
LV
OSC
V+
CAP-
CAP+
VOUT
C1
10u
U4C
LM358
U4B
LM358
C2
R2
10k
R_Photo
U4A
LM358
AD
ELEE4230E
Sheet: /
File: Sensors_PCB.kicad_sch
Title: Sourdough Tracker

Filtering and Kalman Implementation

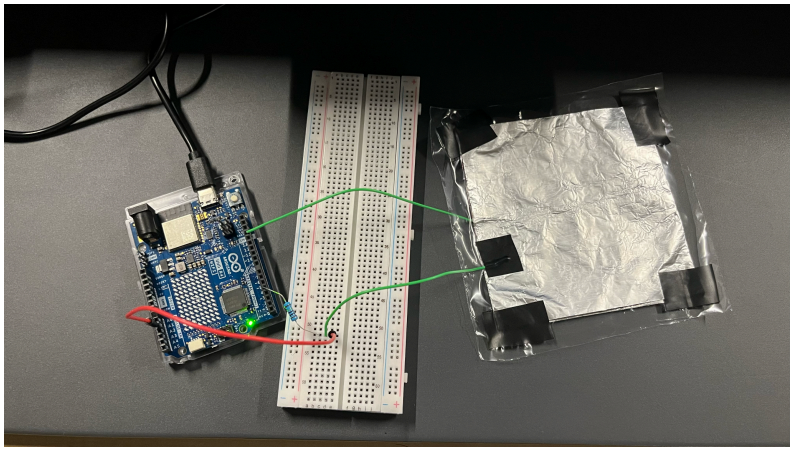


The graph shows the raw calculated lux with large spikes (I waved my hand randomly in front of the sensors) and the Kalman filtered lux that "smoothes" it out while keeping an accurate baseline. The raw signal jumps by several hundred lux within a single sample during the hand-wave, while the filtered trace stays within a narrow band around the true baseline and only drifts toward the disturbance gradually, confirming the filter rejects short transients without erasing genuine slow trends.

A 1D scalar Kalman filter is used, applied independently to two channels: the normalized optical opacity signal and the PCT2075 temperature reading. Each treats the true underlying value as approximately constant between samples (a static, zero-motion model) rather than trying to predict a rate of change, since both opacity and temperature evolve slowly relative to the sampling rate — any fast jump is far more likely to be noise (a hand, a shadow, electrical ripple) than real signal. The filter's structure follows the standard predict/update cycle:

Predict: the prior estimate is carried forward unchanged (x_pred = x_est), and its uncertainty grows by the process noise Q (P_pred = P + Q).
Update: a Kalman gain K = P_pred / (P_pred + R) weighs the new raw reading against the prediction; the estimate is nudged toward the raw measurement by that gain (x_est = x_pred + K*(raw − x_pred)), and the uncertainty is shrunk accordingly (P = (1−K)*P_pred).

4. Capacitive Sensor (User Input System)



My homemade capacitor is made up of 2 10cm x 10cm tinfoil sheets as parallel plates with a 12cm x12cm plastic (ziploc bag) sheet in between as the dielectric.

The fundamental operation behind this is governed by Gauss's Law for electrostatics. Assuming an ideal charge distribution on two parallel conductive plates with a uniform surface area (A) and a separation distance (d), the electric field (E) confined between the plates is constant: $E = Q / (\varepsilon \cdot A)$

Integrating the uniform electric field across the dielectric displacement thickness (d) yields the potential difference (V) between the aluminum foil electrodes: $V = E \cdot d = (Q \cdot d) / (\varepsilon \cdot A)$

Substituting this expression into the fundamental definition of capacitance ($C = Q / V$) yields the classical theoretical transfer function for a parallel-plate capacitor: $C = \varepsilon r \cdot \varepsilon 0 \cdot (A / d)$

- $\varepsilon 0$: Permittivity of free space (8.854×10^{-12} F/m)
- εr: Relative permittivity of the plastic film (≈ 2.26)
- A: Active plate overlapping area ($0.1 \text{ m} \times 0.1 \text{ m} = 0.01 \text{ m}^2$)
- d: Nominal thickness of the plastic film ($\approx 50 \text{ μm} = 5.0 \times 10^{-5}$ m)

$C_{theoretical} = (2.26) \cdot (8.854 \times 10\text{-}12 \text{ F/m}) \cdot [0.01 \text{ m}^2 / (5.0 \times 10^{-5} \text{ m})] \approx 4.002 \text{ nF} = 4002 \text{ pF}$

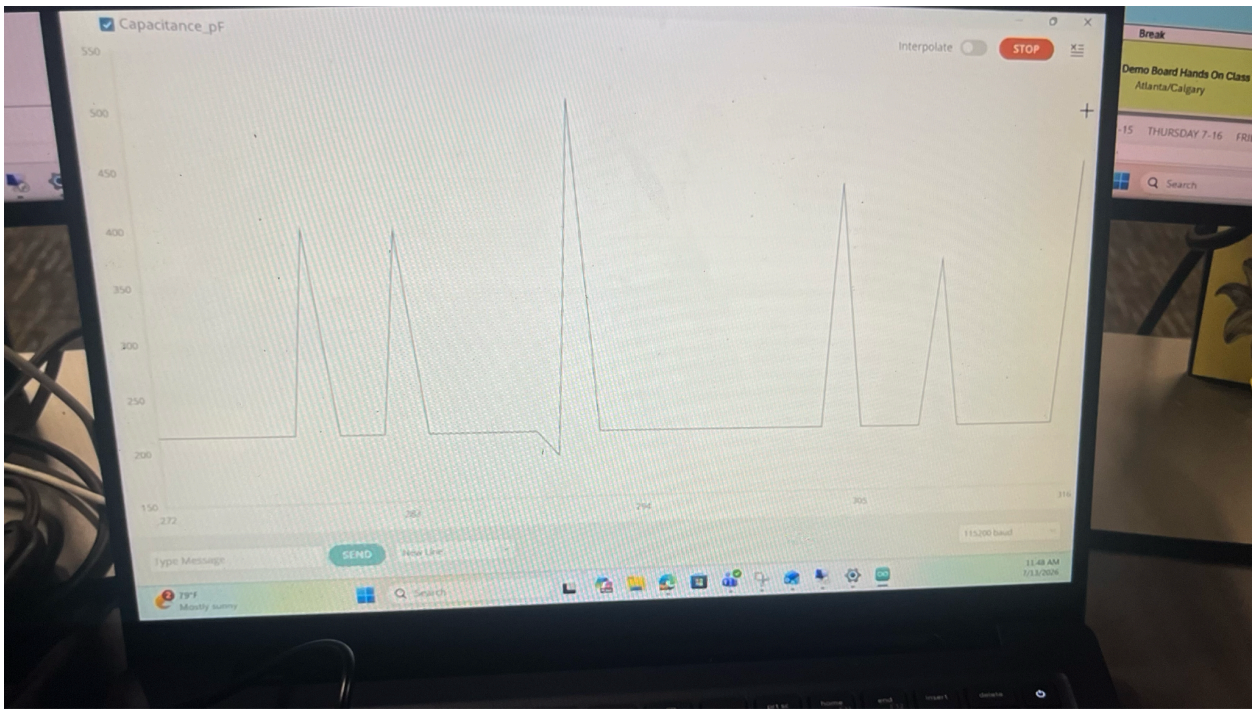
Because this hardware functions as a displacement force sensor, real-world capacitance maps as a non-linear rational function of applied pressure (P). Applied pressure forces the plates closer together, reducing the nominal dielectric thickness by a variable displacement factor (Δd). The experimental transfer function is modeled as: $C_{experimental}(P) = [\varepsilon r \cdot \varepsilon 0 \cdot A / (d0 - f(P))] + C_{parasitic}$ where d0 is the uncompressed resting distance, f(P) is the mechanical compression profile under force, and $C_{parasitic}$ accounts for background circuit layout capacitance.

In reality, hand-cut foil has microscopic wrinkles that trap air pockets at rest, suppressing the initial capacitance to around 3 nF instead of the theoretical 4.002 nF target. Applying pressure flattens the surfaces, and reduces the plate separation distance. Because capacitance is inversely proportional to distance, compressing the sensor drives the measurement upward to about 4 nF, exemplifying the parallel-plate equation under force.

Signal Processing:
To convert the transient response of the homemade sensor into a real-time capacitance measurement, the system uses a software-driven, time-domain RC time constant estimator.

The Arduino first drives the charging digital pin LOW to fully discharge the capacitor. It then switches the charging pin HIGH to apply a 5V step input and immediately starts a microsecond stopwatch. The signal processing pipeline uses high-speed inline polling to monitor the 10-bit ADC pin.

Because a capacitor charges to 63.2% of its steady-state voltage at one time constant, the code holds execution until the analog pin reaches 646 out of 1023 counts. Once this voltage threshold is cleared, the clock stops, and the absolute capacitance is calculated in software by isolating C with the function: $C = \tau / R$



System Integration

What it controls: The capacitive sensor functions as the system's zero/reset input; it establishes the baseline "unrisen" opacity reading at the start of a proof. Pressing the plates together (flattening the foil, reducing plate separation) is the physical gesture the user performs to tell the system "the dough is in the jar now, start tracking from here."

How it interacts with other sensors/logic: When the RC time-constant estimator detects the discharge-to-63.2%-threshold crossing (646/1023 ADC counts) faster than the resting baseline predicts, the algorithm treats this as a deliberate reset event rather than sensor noise. That event does two things in the software architecture: it re-zeroes the LDR opacity baseline that the Kalman filter treats as "ground truth" (rather than the filter continuing to smooth against a possibly stale reference), and it restarts the go-beyond slope-tracking algorithm so peak-detection

begins counting from a fresh batch rather than carrying over slope history from a prior bake. Because the charge/discharge polling routine briefly occupies the main loop, it's scheduled so it doesn't collide with the periodic LDR/ambient/temperature sampling cycle.

What it triggers: A confirmed press triggers three things in sequence: (1) baseline zeroing of the opacity channel, (2) a restart of the peak-detection algorithm's internal state, and (3) a new "proof start" timestamp written to the IoT log, so each logged dataset corresponds to one discrete proofing cycle rather than one continuous, ambiguous data stream.

Limitations and Performance

Limitation: The sensor's baseline capacitance isn't stable since it is sensitive to how the foil happens to be sitting at any given moment, and it will drift further with minor changes in the plastic dielectric over time. Because the reset logic depends on distinguishing a genuine press from this baseline drift, there's a risk of false triggers (drift crossing the threshold on its own) or missed triggers (a light or uneven press not fully flattening the foil). The sensor doesn't have the datasheet-guaranteed sensitivity floor that a pre-packaged touch IC would provide.

Observations during testing: Measured resting capacitance landed around 3nF rather than the theoretical 4002pF, consistent with air gaps from the hand-cut foil rather than a true 50μm uniform gap. Under compression, the measured value rose to roughly 4nF, tracking the expected inverse relationship between capacitance and plate separation, though the exact magnitude varied depending on how much force was applied by hand.

Calibration/thresholding used: The RC timing method itself was validated against a known reference capacitor before trusting readings from the foil sensor, confirming the $\tau = RC$ extraction logic was sound independent of the sensor's own inconsistency. For the reset trigger itself, the threshold was set empirically between the observed resting range (~3nF) and compressed range (~4nF) rather than at the theoretical midpoint, since the real baseline consistently sat below the theoretical prediction — this threshold was tuned by observing repeated press/release cycles rather than calculated purely from the parallel-plate equation.

5. Other Sensors Used

VEML7700 - Ambient Light Sensor

Raw physics: A photodiode inside the sensor generates a photocurrent proportional to incident light intensity across roughly the visible spectrum, weighted to mimic human eye sensitivity rather than raw irradiance.

Output characteristics: 16-bit ADC resolution internally, with a stated sensitivity down to 0.0036 lux/count. In this project, the sensor isn't used near its low-light floor or its saturation ceiling. It operates in the everyday kitchen-lighting range (roughly single digits of lux in a dim room up to a few hundred lux under overhead lighting or near a window), which sits comfortably within its usable range without needing gain/integration-time adjustment for extreme conditions.

Expected vs. actual behavior: The datasheet models the photodiode response as roughly linear with true lux. What was actually characterized, though, wasn't lux vs. an absolute light meter. It was the sensor's output as a function of physical distance from a fixed light source (a phone flashlight stepped from 12 inches down to 1 inch). That relationship is expected to fall off with the inverse-square law, and the fitted transfer function ($I = 3495 \cdot d^{-1.1}$) landed close to that expectation, with the exponent slightly off from a clean −2, likely due to the flashlight not behaving as an ideal point source at short range and minor hand-shake misalignment during data collection. So the sensor's own linearity wasn't really the variable under test here. The optical geometry of the light source was, and it behaved close to, but not exactly like, the idealized model.

Internal signal processing: The photodiode current almost certainly passes through an internal transimpedance amplifier to produce a voltage, which is then digitized by the onboard ADC and scaled in firmware against a known lux-per-count constant before being exposed over I2C. None of this is user-accessible, but the clean monotonic register output is consistent with that kind of internal current-to-digital pipeline rather than a raw unconditioned analog output.

Limitation: At a fixed light position, the readings weren't perfectly constant. Repeated samples at one distance showed measurable output fluctuation, meaning the sensor carries enough short-term noise that a single instantaneous reading can't be trusted as ground truth. This is part of the motivation for treating this sensor as an input to a filter rather than a standalone measurement.

PCT2075 - Temperature Sensor

Raw physics: A silicon bandgap temperature sensing element produces a voltage that varies predictably and linearly with junction temperature, exploiting the known temperature dependence of a diode's forward voltage drop.

Output characteristics: 11-bit two's-complement digital output at 0.125°C resolution, with a datasheet-guaranteed accuracy of ±1°C across −25°C to +100°C, far wider than this project's actual operating band, which only ever spans kitchen-ambient to warm proofing conditions (roughly 0°C to 50°C in testing).

Expected vs. actual behavior: Across seven calibration points from ice water to hot tap water, the sensor tracked the reference kitchen thermometer closely, but consistently read slightly high, by about 0.3 to 0.7°C, with the gap growing modestly at the higher end (0.4°C high at 0°C, 0.6°C high at 49°C). That's expected behavior within spec (the datasheet only guarantees ±1°C), but the consistent positive-offset direction, rather than random scatter above and below the reference, suggests a systematic bias. Possibly self-heating from the sensor's own operation, the Ziploc bag layer adds a small thermal lag, or a fixed calibration offset in this particular unit, rather than pure measurement noise.

Internal signal processing: The analog bandgap voltage is digitized on-chip by what's most likely a sigma-delta or successive-approximation ADC tuned for the sensor's expected voltage range, then converted to a temperature value in two's-complement format and buffered in an internal register for I2C readout. This matches the clean, quantized 0.125°C steps observed rather than continuous analog noise.

Limitation: The sensor's silicon die is exposed to the same enclosure air as the rest of the electronics, so any airflow or heat radiating from the MCU/regulator side of the board, even with physical separation on the layout, can still bias the reading. The consistent positive offset seen in calibration may partly reflect this rather than a diode-level fabrication error. That offset was corrected for in software after calibration, but only for the tested range, so accuracy outside 0 to 50°C hasn't been verified in this project's context.

6. Digital Logic + Algorithms

Algorithm 1: Optical Processing

Sensor Reading Functions: Reads raw analog light transmission data from the LDR circuit via the ADC pin and polls digital light values over I2C from the VEML7700. It scales these inputs into a relative opacity metric. We need a direct physical measurement of how much light passes through or reflects off the starter to quantify dough density changes over time.

Kalman Filter: Takes the raw optical values and runs them through a 1D linear Kalman filter to smooth out incoming signal spikes.Room light flickers, shadows, and power supply ripple create noisy optical readings. The Kalman filter strips this noise without adding phase lag like a standard moving average would.

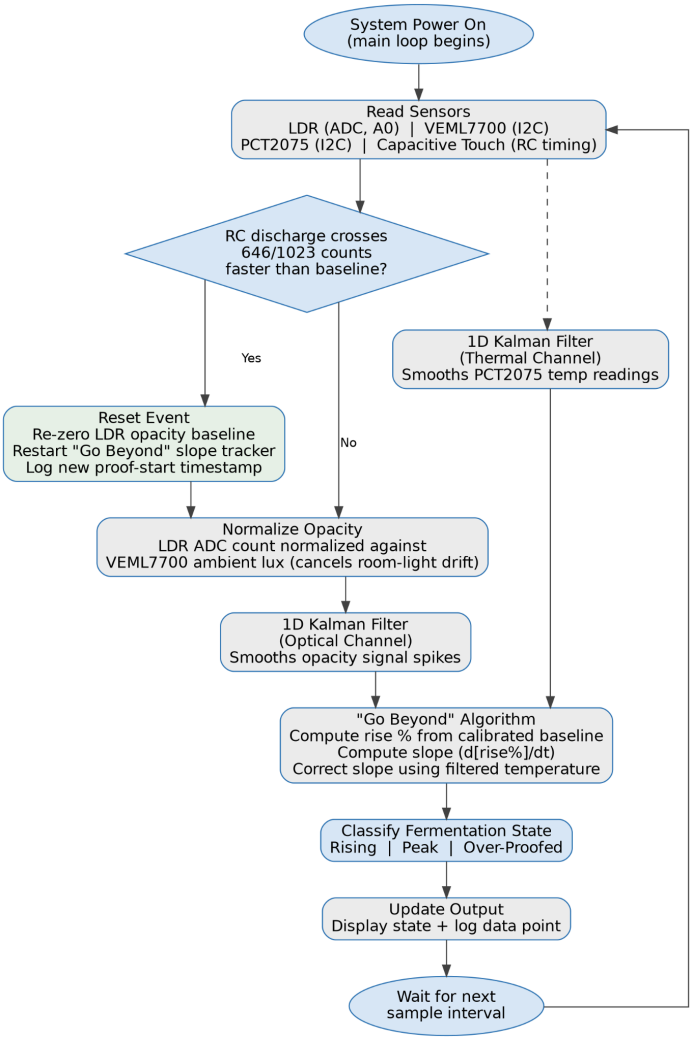
"Go Beyond" Feature: Computes the derivative of opacity over time to track the rate of light attenuation shift rather than relying on a static cutoff value. Since different flour types and hydration levels scatter light differently, calculating the rate of change lets the system adaptively detect expansion activity regardless of the starter recipe.

Algorithm 2: Thermal & Fermentation State Tracking

Sensor Reading Functions: Queries readings from the PCT2075 temperature sensor over I2C and converts the 11-bit two's-complement readout into degrees Celsius. Yeast activity is heavily dependent on ambient temperature, so we need continuous thermal monitoring to contextualize rise time.

Kalman Filter: Filters thermal reading drift and sudden localized micro-temperature fluctuations around the jar wall. External drafts or temporary handling can cause rapid surface temp jumps. Filtering ensures our thermal model reflects actual starter culture conditions rather than quick spikes.

"Go Beyond" Feature: Combines the filtered thermal data with optical growth slope trends to dynamically calculate metabolic activity and update the progress bar state ("Rising", "Ready", or "Over-Proofed") on the output. Temperature directly changes yeast metabolic rates (warmer = faster peak, cooler = slower rise). Fusing temp data with growth rate gives an accurate prediction of peak activity instead of just guessing based on height alone.



Design Decision: Microcontroller Selection

Chosen*:* Arduino Uno R4 Wifi. The R4's 32-bit Renesas RA4M1 core runs the floating-point Kalman filter updates and the "Go Beyond" slope/temperature-correction math every sample without the latency an 8-bit core would add, and it drives the microsecond-resolution RC-timing loop the capacitive sensor depends on for a stable $\tau = RC$ estimate. Its single I2C bus comfortably hosts both the VEML7700 and PCT2075 without contention, and its onboard 5V-tolerant ADC pairs directly with the LM358 buffer output, so no extra level-shifting or interfacing circuitry was needed.

Alternative considered: ESP32 development board.
Why it wouldn't work as well*:* The ESP32 draws too much power for a multi-hour, battery-only proof and would've required redesigning the analog interfacing around its 3.3V logic instead of the R4's 5V-tolerant pins.

7. Final Design and Experimental Results

Result 1: PCT2075 Temperature Calibration

Reference Description	Reference Temp (°C)	PCT2075 Reading (°C)
Ice water	0.0	0.4
Cold tap + ice water	10.0	10.3
Cold tap water	17.0	17.2
Room temperature	23.0	23.3
Warm tap water	31.0	31.4
Warm + hot tap water	40.0	40.5
Hot tap water	49.0	49.6

The offset falls well within the sensor's ±1°C datasheet tolerance, but its consistent direction indicates a bias rather than measurement noise. This was considered when integrating temperature data in the go-beyond algorithm.

Result 2: Kalman Filter Transient Rejection



Over the recorded test window, the raw calculated lux signal swung from near 0 up to a peak of roughly 1,555 lux, driven by large instantaneous spikes during a deliberate hand-wave disturbance test. The Kalman-filtered output covered a narrower range, roughly 80 to 1,165 lux over the same window with the filtered trace visibly lagging and smoothing over the spike rather than tracking it directly.

Result 3: LDR-to-Lux Regression Accuracy

Timestamp	Lux at 6 inches
20:50:04	519.49
20:50:05	514.02
20:50:05	511.72
20:50:06	515.64
20:50:06	513.96
20:50:07	517.94
20:50:07	519.21
20:50:08	518.63
20:50:08	511.83
20:50:09	515.52
20:50:09	515.87
20:50:10	517.54
20:50:10	517.59
20:50:11	516.67
20:50:11	518.75
20:50:12	520.47

A ±1.39 lux uncertainty band on a signal that needs to detect meaningful, sustained shifts of tens to hundreds of lux as dough rises is small enough to be negligible, confirming that under static conditions, sensor and environmental noise alone won't be mistaken for real fermentation activity. This directly supports the CP1 requirement for an accurate, non-invasive optical measurement: the baseline noise floor is well below the signal of interest, so the rise-detection algorithm isn't chasing measurement jitter.

8. Failure Analysis

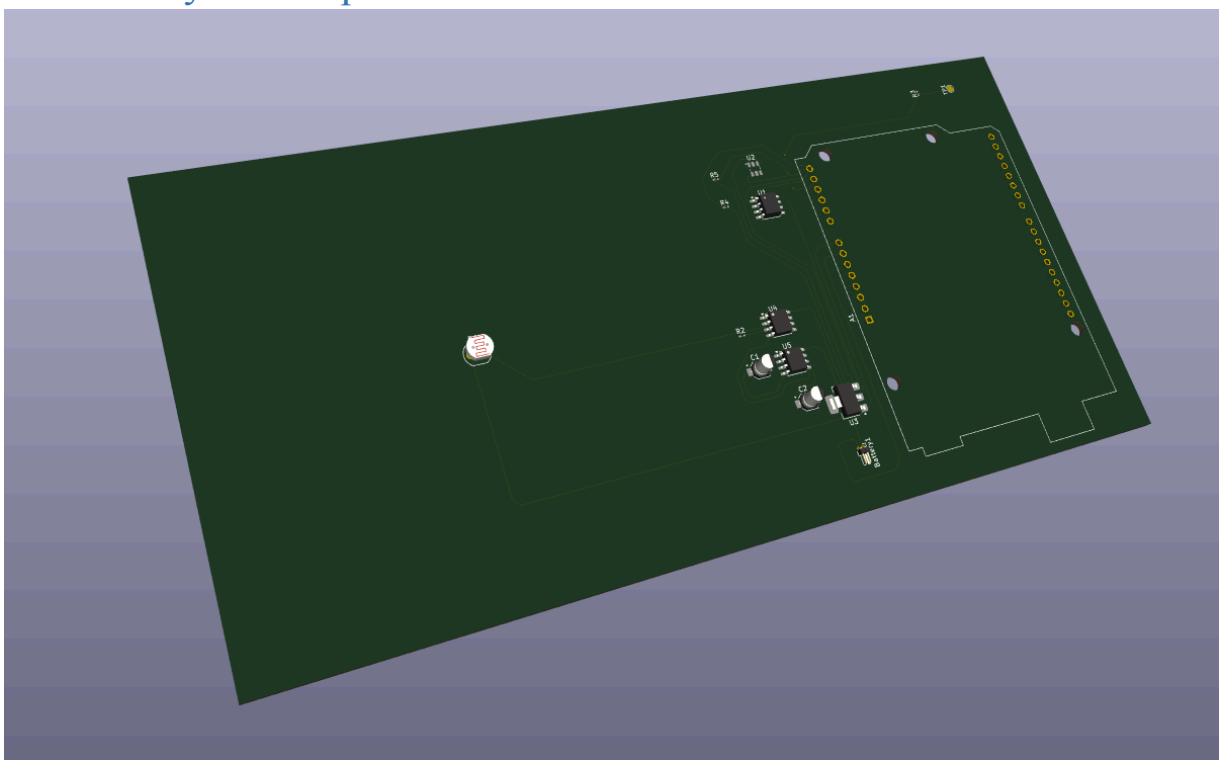
What I expected: Since the VEML7700 ambient light sensor and PCT2075 temperature sensor use fixed, distinct I2C addresses, I expected them to coexist on the same bus with no conflict, especially since I2C is designed specifically to support multiple devices sharing two shared lines (SDA/SCL).

What actually happened: When I wired the second I2C sensor onto the bus alongside the first, the first sensor stopped returning valid readings. It looked like the sensor disappeared from the bus entirely.

Why my assumption failed: I had assumed that because the two sensors had different addresses, wiring them in parallel was purely a software/addressing problem. What I hadn't accounted for is that most breakout boards ship with their own onboard pull-up resistors on SDA/SCL. Adding a second board in parallel effectively puts two sets of pull-ups in parallel, which lowers the net resistance on the bus below the recommended range. This weakens the logic-high voltage level and degrades signal integrity enough that the bus can misread or drop communication with devices already on it.

What I changed: I added a set of 10k pull-up resistors from 3.3V to SDA and SCL on the Arduino in junction with the sensor connections. I also made the wires as short as I could between connections. After this change, both sensors reported consistent, simultaneous readings on the shared bus.

9. Physical Implementation



For the PCB design, I wanted to keep things clean and make some untraditional mechanical adjustments to fit a jar. The biggest limitation was to make space for a jar to fit within the system and accurately hold measurements. I kept the power supply parts including the AMS1117 regulator and ICL7660 inverter, separated from the analog side where the LDR comes in, mostly so any switching noise wouldn't mess with my ADC readings. For the PCT2075 temp sensor, I routed those I2C traces as short and direct as possible to avoid capacitance issues on the bus. Instead of just shoving loose jumper wires into standard headers, I used JST-XH connectors for all the off-board sensor hookups. They snap in really securely, so nothing accidentally wiggles loose when placing and removing jars constantly. I also kept the LDR and LED connections grouped together on their own part of the layout, which made it way easier to run the wires straight to the jar without interfering with anything else.

Power Implementation:

My setup includes 4 AA batteries to create 6.4V to power the Arduino through its Vin pin while using a 3.3V LDO to step down voltage safely for the sensors. I also use an inverter to achieve a -3.3V for op-amps.

8. Course Reflection

Project Reflections

1. What assumption was most incorrect?
Probably the assumption that the parallel-plate capacitor equation would predict my homemade sensor's behavior closely. The theoretical model predicted a resting capacitance of about 4.002 nF, but the actual hand-cut foil sensor sat closer to 3 nF at rest.

2. Where did theory deviate most from reality?
Also the capacitive sensor. The parallel-plate formula assumes a perfectly uniform dielectric thickness, but hand-cut foil has microscopic wrinkles that trap air pockets, which suppresses the baseline capacitance well below the theoretical target. The direction of the shift (measured lower than predicted) was consistent and repeatable, which told me it was a structural/build issue rather than measurement noise.

3. What design decision limited your performance?
The need to keep each sensor physically separated on the board layout to get accurate readings — particularly isolating the PCT2075 from the MCU/regulator to avoid self-heating bias. This constrained how compact the final layout could be.

4. What would you NOT do again?
Calibrate the sensors with variables that did not match my project’s environment. While the methods I used to calibrate my sensors allowed me to test their ranges, they didn’t help pinpoint the more specific range I was seeing in my system.

Lecture Reflections

I think they provided a good understanding of fundamentals rather than circuits/math. It felt like a physics class, which I actually enjoyed more than solving problems.

9. References

Human References:

Youtubers for PCB review

AI References:

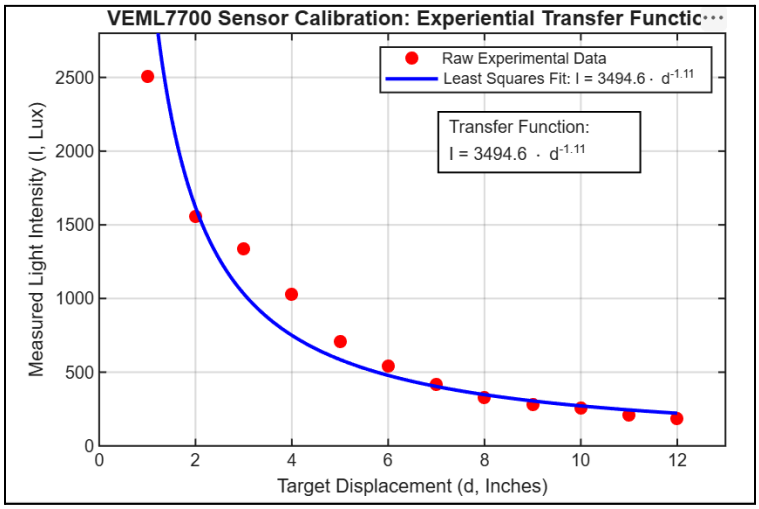
- **Tool used:** Copilot
  Prompt: Generate image of final revision idea of LED output
  What was wrong: Explain clearly what inputs/outputs, where the LED and sensors are located

- **Tool used:** Claude
  Prompt: Combine these scripts into one algorithm
  What was wrong: Not understanding the use of the light sensor to check the LDR with the Kalman algorithm

- **Tool used:** Claude
  Prompt: Create a flowchart that details the final code
  What was wrong: Specific phrases to include/ what I wanted it to look like

Appendix

Ambient light sensor data



Distance (in)	Illuminance (lux)
12	184
11	207
10	258
9	283
8	325
7	418
6	539
5	709
4	1026
3	1336
2	1554
1	2508

Go Beyond Code (C++ ino):

```
#include <Wire.h>
#include <Adafruit_VEML7700.h>

// ---------------------------------------------------------------------------
// Pin assignments
// ---------------------------------------------------------------------------
const uint8_t PIN_LDR_ADC       = A0;  // buffered photoresistor divider output
const uint8_t PIN_CAP_CHARGE    = 7;   // digital output: drives capacitor charge/discharge
const uint8_t PIN_CAP_SENSE     = A1;  // analog input: monitors capacitor voltage

// ---------------------------------------------------------------------------
// PCT2075 register map (no dedicated Arduino library needed — it's simple)
// ---------------------------------------------------------------------------
const uint8_t PCT2075_ADDR      = 0x48; // default address, confirm against your wiring (A0-A2 pins)
const uint8_t PCT2075_REG_TEMP  = 0x00;

Adafruit_VEML7700 veml = Adafruit_VEML7700();

// ---------------------------------------------------------------------------
// LDR -> Lux regression (from your CP6 calibration, R^2 = 0.91)
//   RLDR = A * Lux^B  =>  inverted for real-time lux from ADC counts
// ---------------------------------------------------------------------------
const float LDR_A = 496.5855f;
const float LDR_B = 0.0403f;
const float DIVIDER_R_OHMS = 10000.0f; // fixed 10k pulled up to 3.3V LDO rail
const float VCC = 3.3f;

// Valid indoor calibration range from your report — outside this, trust the
```

```
// value less (per your RMSE discussion, error grows above ~4000 lux).
const float LDR_LUX_MIN_VALID = 243.0f;
const float LDR_LUX_MAX_VALID = 7178.0f;

// ---------------------------------------------------------------------------
// Capacitive sensor (RC time-constant reset trigger)
// ---------------------------------------------------------------------------
const int   CAP_THRESHOLD_COUNTS = 646;      // 63.2% of 1023, per your report
const float CAP_CHARGE_R_OHMS    = 1000000.0f; // CALIBRATE ME: your actual charging resistor value
unsigned long capTimeoutMicros   = 200000UL;  // safety timeout so a bad press can't hang the loop

// ---------------------------------------------------------------------------
// Kalman filter state (1D scalar, static/zero-motion model — matches Appendix)
// ---------------------------------------------------------------------------
struct KalmanState {
  float x_est;   // current estimate
  float P;       // current estimate uncertainty
  float Q;       // process noise (tune: higher = trusts new readings more)
  float R;       // measurement noise (tune: higher = trusts prior estimate more)
};

KalmanState opacityFilter = { 0.0f, 1.0f, 0.5f, 4.0f };   // CALIBRATE ME: Q/R for LDR channel
KalmanState tempFilter    = { 0.0f, 1.0f, 0.01f, 1.0f };  // CALIBRATE ME: Q/R for PCT2075 channel
bool filtersInitialized = false;

float kalmanUpdate(KalmanState &kf, float rawMeasurement) {
  // Predict
```

```
  float x_pred = kf.x_est;
  float P_pred = kf.P + kf.Q;
  // Update
  float K = P_pred / (P_pred + kf.R);
  kf.x_est = x_pred + K * (rawMeasurement - x_pred);
  kf.P = (1.0f - K) * P_pred;
  return kf.x_est;
}

// ---------------------------------------------------------------------------
// "Go Beyond" fermentation-tracking state
// ---------------------------------------------------------------------------
enum FermentState { RISING, PEAK, OVER_PROOFED };

float baselineOpacity      = 0.0f;   // set at reset: "flat dough" reference
float fullyRisenOpacity    = 0.0f;   // CALIBRATE ME: your fully-risen reference reading
float lastRisePercent      = 0.0f;
unsigned long lastSampleMs = 0;
float riseSlopePerSec      = 0.0f;

const float TEMP_REFERENCE_C   = 24.0f; // "room temp" reference for slope correction
const float TEMP_CORRECTION_K  = 0.06f; // CALIBRATE ME: fractional rate change per degree C

const float PEAK_RISE_PERCENT       = 95.0f; // considered "at peak" above this
const float OVERPROOF_SLOPE_DROP    = -0.5f; // %/sec — negative slope after peak = collapsing structure

FermentState currentState = RISING;
```

```
unsigned long proofStartMillis = 0;

// ---------------------------------------------------------------------------
void setup() {
  Serial.begin(115200);
  Wire.begin();

  pinMode(PIN_CAP_CHARGE, OUTPUT);
  digitalWrite(PIN_CAP_CHARGE, LOW);

  if (!veml.begin()) {
    Serial.println("ERROR: VEML7700 not found on I2C bus.");
  }

  Serial.println("Sourdough Tracker booted. Press capacitive sensor to start a proof.");
}

// ---------------------------------------------------------------------------
void loop() {
  // 1) Check for a reset/start-of-proof press (non-blocking-ish, bounded by timeout)
  if (checkCapacitivePress()) {
    handleProofReset();
  }

  // 2) Regular sampling cycle (runs independent of the capacitive check)
  static unsigned long lastLoopMs = 0;
  unsigned long now = millis();
  if (now - lastLoopMs >= 1000) { // 1 Hz sample rate — adjust as needed
    lastLoopMs = now;
    runSensorCycle(now);
```

```
  }
}

// ---------------------------------------------------------------------------
// Capacitive sensor: RC time-constant estimator
// Returns true if a press (fast RC charge) was detected.
// ---------------------------------------------------------------------------
bool checkCapacitivePress() {
  // Fully discharge
  digitalWrite(PIN_CAP_CHARGE, LOW);
  delayMicroseconds(50);

  // Start charging and time until threshold is crossed
  digitalWrite(PIN_CAP_CHARGE, HIGH);
  unsigned long startMicros = micros();
  unsigned long elapsed = 0;
  bool crossed = false;

  while (elapsed < capTimeoutMicros) {
    if (analogRead(PIN_CAP_SENSE) >= CAP_THRESHOLD_COUNTS) {
      crossed = true;
      break;
    }
    elapsed = micros() - startMicros;
  }

  digitalWrite(PIN_CAP_CHARGE, LOW); // discharge again for next cycle

  if (!crossed) return false;
```

```
  float tau_seconds = elapsed / 1e6f;
  float capacitance_F = tau_seconds / CAP_CHARGE_R_OHMS; // C = tau / R

  // A "press" is a deliberate, distinctly faster/slower charge time than the
  // resting baseline predicts. CALIBRATE ME: tune this threshold against your
  // own observed resting (~3nF) vs compressed (~4nF) range from testing.
  const float PRESS_CAPACITANCE_THRESHOLD_F = 3.5e-9f;
  return capacitance_F >= PRESS_CAPACITANCE_THRESHOLD_F;
}

// ---------------------------------------------------------------------------
void handleProofReset() {
  Serial.println(">>> Reset triggered: zeroing baseline, restarting Go Beyond tracking.");

  float rawLux = readLdrLux();
  baselineOpacity = rawLux;
  lastRisePercent = 0.0f;
  riseSlopePerSec = 0.0f;
  currentState = RISING;
  proofStartMillis = millis();

  // Re-seed the Kalman filter so it doesn't smooth against a stale reference
  opacityFilter.x_est = rawLux;
  opacityFilter.P = 1.0f;
  filtersInitialized = true;

  logProofStart(proofStartMillis);
}

// ---------------------------------------------------------------------------
```

```
void runSensorCycle(unsigned long nowMs) {
  // --- Read raw sensors ---
  float rawLux = readLdrLux();
  float ambientLux = veml.readLux();
  float tempC = readPct2075TempC();

  if (!filtersInitialized) {
    opacityFilter.x_est = rawLux;
    tempFilter.x_est = tempC;
    filtersInitialized = true;
  }

  // --- Kalman filter both channels ---
  float filteredOpacity = kalmanUpdate(opacityFilter, rawLux);
  float filteredTempC   = kalmanUpdate(tempFilter, tempC);

  // --- Go Beyond: normalize against ambient light to cancel room-light drift ---
  float normalizedOpacity = filteredOpacity - ambientLux;

  // --- Convert to rise percentage using baseline / fully-risen calibration ---
  float risePercent = 0.0f;
  if (fullyRisenOpacity != baselineOpacity) {
    risePercent = 100.0f * (normalizedOpacity - baselineOpacity) /
                  (fullyRisenOpacity - baselineOpacity);
    risePercent = constrain(risePercent, 0.0f, 150.0f); // allow some over-range for over-proof
detection
  }

  // --- Rate of change (slope), temperature-corrected ---
  float dtSec = (lastSampleMs == 0) ? 1.0f : (nowMs - lastSampleMs) / 1000.0f;
```

```
  float rawSlope = (risePercent - lastRisePercent) / dtSec;
  float tempCorrection = 1.0f + TEMP_CORRECTION_K * (filteredTempC - TEMP_REFERENCE_C);
  riseSlopePerSec = rawSlope * tempCorrection;


  lastRisePercent = risePercent;
  lastSampleMs = nowMs;


  // --- Classify fermentation stage ---
  currentState = classifyState(risePercent, riseSlopePerSec);


  logSample(nowMs, rawLux, ambientLux, filteredTempC, risePercent, riseSlopePerSec, currentState);
}


// ---------------------------------------------------------------------------
FermentState classifyState(float risePercent, float slopePerSec) {
  if (risePercent >= PEAK_RISE_PERCENT && slopePerSec <= OVERPROOF_SLOPE_DROP) {
    return OVER_PROOFED;
  }
  if (risePercent >= PEAK_RISE_PERCENT) {
    return PEAK;
  }
  return RISING;
}


// ---------------------------------------------------------------------------
// Sensor read helpers
// ---------------------------------------------------------------------------
float readLdrLux() {
```

```
  int adcCount = analogRead(PIN_LDR_ADC);
  float vOut = (adcCount / 1023.0f) * VCC;

  // Guard against divide-by-zero as vOut approaches VCC
  if (vOut >= VCC - 0.001f) vOut = VCC - 0.001f;

  // Voltage divider: LDR on top, fixed 10k on bottom, output vOut
  float rLdr = DIVIDER_R_OHMS * (VCC - vOut) / vOut;

  // Invert RLDR = A * Lux^B  =>  Lux = (RLDR / A) ^ (1/B)
  float lux = pow(rLdr / LDR_A, 1.0f / LDR_B);

  return lux; // caller/log can flag if outside LDR_LUX_MIN_VALID/MAX_VALID
}

float readPct2075TempC() {
  Wire.beginTransmission(PCT2075_ADDR);
  Wire.write(PCT2075_REG_TEMP);
  Wire.endTransmission(false);
  Wire.requestFrom(PCT2075_ADDR, (uint8_t)2);

  if (Wire.available() < 2) {
    return NAN; // I2C read failed
  }

  uint8_t msb = Wire.read();
  uint8_t lsb = Wire.read();

  // 11-bit two's complement, left-justified in a 16-bit word, 0.125 C/LSB
  int16_t raw = ((int16_t)(msb << 8 | lsb)) >> 5;
```

```
  float tempC = raw * 0.125f;
  return tempC;
}

// ---------------------------------------------------------------------------
// Logging
// ---------------------------------------------------------------------------
void logProofStart(unsigned long timestampMs) {
  Serial.print("PROOF_START,");
  Serial.println(timestampMs);
}

void logSample(unsigned long t, float rawLux, float ambientLux, float tempC,
               float risePercent, float slope, FermentState state) {
  Serial.print(t); Serial.print(",");
  Serial.print(rawLux, 2); Serial.print(",");
  Serial.print(ambientLux, 2); Serial.print(",");
  Serial.print(tempC, 2); Serial.print(",");
  Serial.print(risePercent, 2); Serial.print(",");
  Serial.print(slope, 3); Serial.print(",");
  switch (state) {
    case RISING:        Serial.println("RISING"); break;
    case PEAK:          Serial.println("PEAK"); break;
    case OVER_PROOFED:   Serial.println("OVER_PROOFED"); break;
  }
}
```

Kalman Filter:

```
  float x_pred = x_est;                          // Predict state ahead
   float P_pred = P + Q;                          // Predict error
   K = P_pred / (P_pred + R);                     // Calculate the
Kalman Gain
   x_est = x_pred + K * (rawCalculatedLux - x_pred); // Update
estimate with real data
   P = (1.0 - K) * P_pred;                        // Update error
```

Temp. Sensor Data

Reference Description	Reference Temp (°C)	PCT2075 Reading (°C)
Ice water	0.0	0.4
Cold tap + ice water	10.0	10.3
Cold tap water	17.0	17.2
Room temperature	23.0	23.3
Warm tap water	31.0	31.4
Warm + hot tap water	40.0	40.5
Hot tap water	49.0	49.6

Videos:

Touch Sensor

Final Video